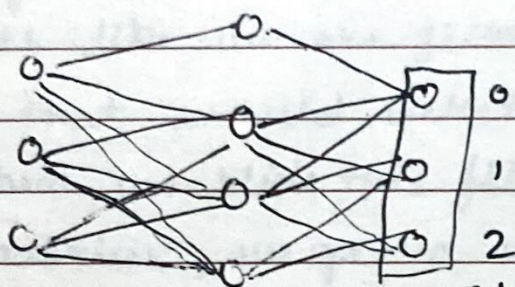


Back propagation



o/p actual	Expected o/p	cost of network total
0	0	$(0-0)^2 = 0$
1	3	$(1-3)^2 = 4$
2	5	$(2-5)^2 = 9$
	3	$- \nabla \text{gradient}$

-ve gradient of each weight and biases tell us how much these weights need to change to minimize the cost.

So, Back propagation is simply adjusting the weights to minimize gradient.

Magnitude of each cost tells how sensitive it is to weights and bias.

$$\textcircled{2} \quad 0.2 = 0 (w_0 a_0 + w_1 a_1 + \dots + w_{n-1} a_{n-1} + b)$$

$a_0 \dots a_{n-1}$ are activations from previous layer

$b = \text{bias}$

$w_i = \text{weights}$

brighter neurons from previous layers have higher activation, co-efficient values

Hence their weights are more sensitive to changes have more influence on cost function

Hebbian theory

11

Neurons that fire together wire together.
Biggest increase in weights happen to neurons that are most active.

So to make a neuron more active depends we need to increase ^{or decrease} the activation co-efficients of previous layers which done by changing weights and biases of its previous layers. This is called backward propagation (of weights)

In the principal of gradient descent we need to do this for all weights and biases for each neuron and for each sample. This is very time intensive.

Hence mini-batch gradient descent or stochastic gradient descent is used.

i.e. Randomly subdivide data into minibatches and compute total weight and then cost for each layer.